UNIT 1

What is Software Engineering?

Software Engineering is the process of designing, developing, testing, and maintaining software. It is a systematic and disciplined approach to software development that aims to create high-quality, reliable, and maintainable software.

1. Software engineering includes a variety of techniques, tools, and methodologies, including requirements analysis, design, testing, and maintenance.
2. It is a rapidly evolving field, and new tools and technologies are constantly being developed to improve the software development process.
3. By following the principles of software engineering and using the appropriate tools and methodologies, software developers can create high-quality, reliable, and maintainable software that meets the needs of its users.
4. Software Engineering is mainly used for large projects based on software systems rather than single programs or applications.
5. The main goal of Software Engineering is to develop software applications for improving quality, budget, and time efficiency.
6. Software Engineering ensures that the software that has to be built should be consistent, correct, also on budget, on time, and within the required requirements.

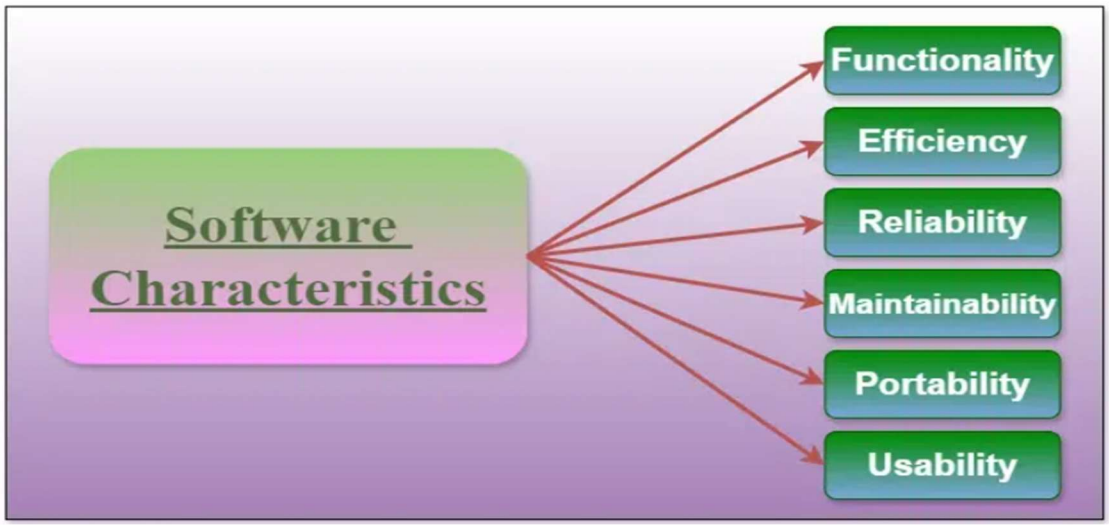
Components of Software

There are three main components of the software:

1. **Program:** A computer program is a list of instructions that tell a computer what to do.
2. **Documentation:** Source information about the product contained in design documents, detailed code comments, etc.
3. **Operating Procedures:** Set of step-by-step instructions compiled by an organization to help workers carry out complex routine operations.

<u>Components of Software Characteristics</u>

There are 6 components of Software Characteristics are discussed here. We will discuss each one of them in detail.



Functionality:

It refers to the degree of performance of the software against its intended purpose.

Functionality refers to the set of features and capabilities that a software program or system provides to its users. It is one of the most important characteristics of software, as it determines the usefulness of the software for the intended purpose. Examples of functionality in software include:

- Data storage and retrieval
- Data processing and manipulation
- User interface and navigation
- Communication and networking
- Security and access control
- Reporting and visualization
- Automation and scripting

The more functionality a software has, the more powerful and versatile it is, but also the more complex it can be. It is important to balance the need for functionality with the need for ease of use, maintainability, and scalability.

Reliability:

A set of attributes that bears on the capability of software to maintain its level of performance under the given condition for a stated period of time.

Reliability is a characteristic of software that refers to its ability to perform its intended functions correctly and consistently over time. Reliability is an important aspect of software quality, as it helps ensure that the software will work correctly and not fail unexpectedly.

Examples of factors that can affect the reliability of software include:

1. Bugs and errors in the code
2. Lack of testing and validation
3. Poorly designed algorithms and data structures
4. Inadequate error handling and recovery
5. Incompatibilities with other software or hardware

To improve the reliability of software, various techniques, and methodologies can be used, such as testing and validation, formal verification, and fault tolerance.

Efficiency:

It refers to the ability of the software to use system resources in the most effective and efficient manner. The software should make effective use of storage space and executive command as per desired timing requirements.

Efficiency is a characteristic of software that refers to its ability to use resources such as memory, processing power, and network bandwidth in an optimal way. High efficiency means that a software program can perform its intended functions quickly and with minimal use of resources, while low efficiency means that a software program may be slow or consume excessive resources.

Examples of factors that can affect the efficiency of the software include:

1. Poorly designed algorithms and data structures
2. Inefficient use of memory and processing power
3. High network latency or bandwidth usage

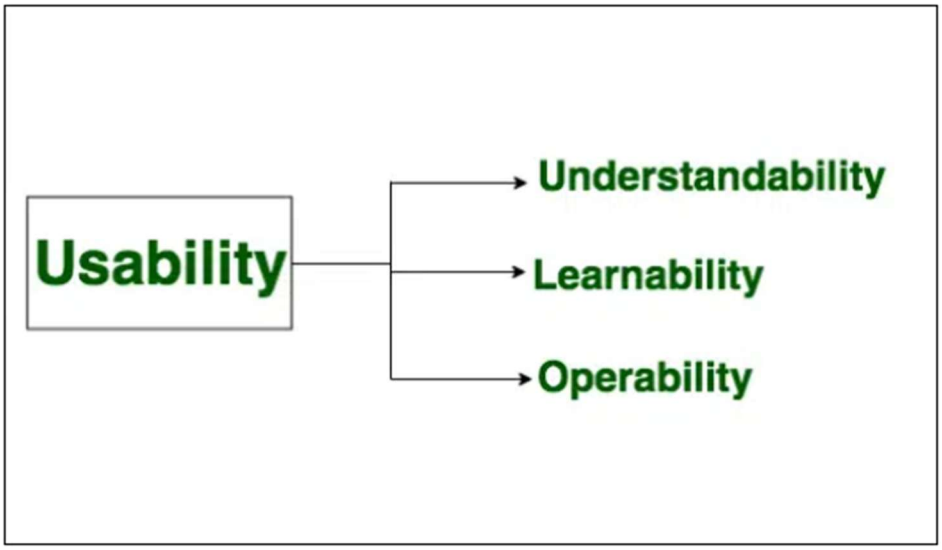
4. Unnecessary processing or computation
5. Unoptimized code

To improve the efficiency of software, various techniques, and methodologies can be used, such as performance analysis, optimization, and profiling.

Usability:

It refers to the extent to which the software can be used with ease. the amount of effort or time required to learn how to use the software.

Required functions are:

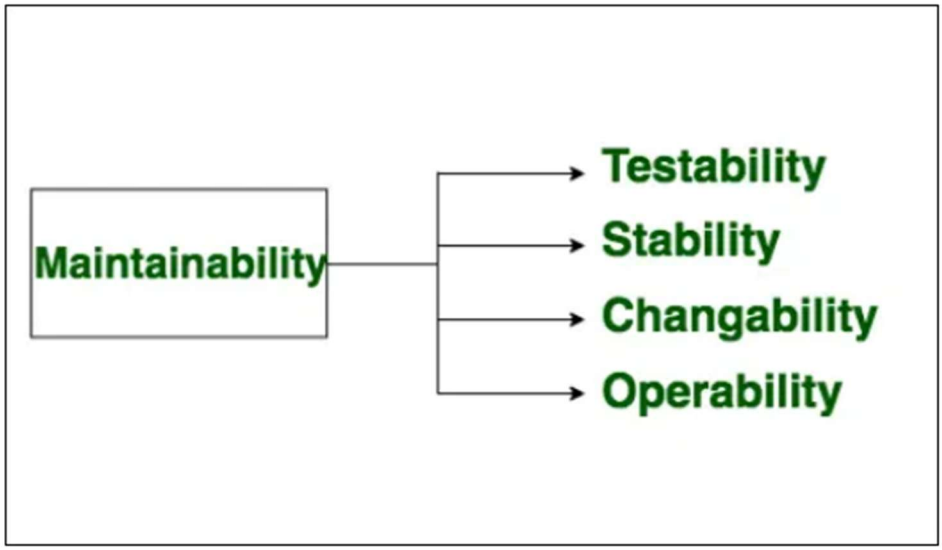


Usability

Maintainability:

It refers to the ease with which modifications can be made in a software system to extend its functionality, improve its performance, or correct errors.

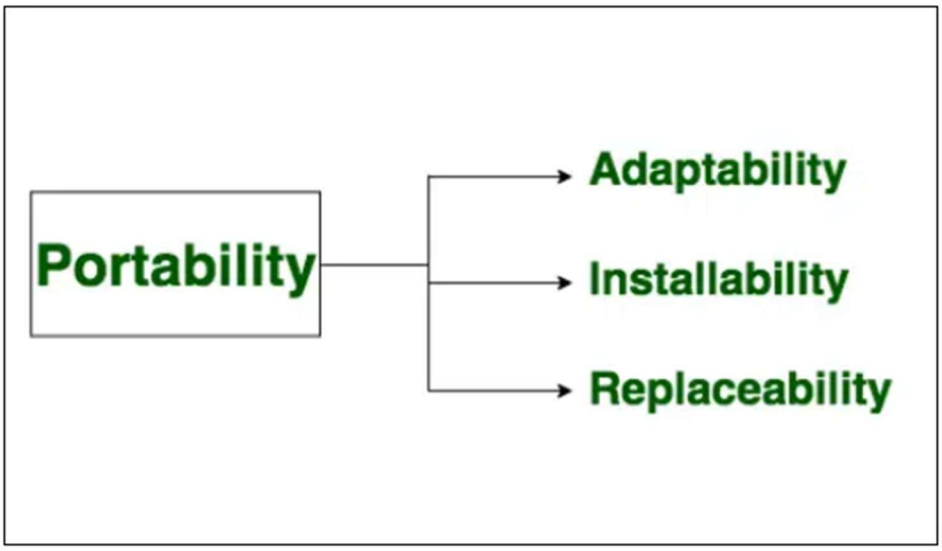
Required functions are:



Maintainability

Portability:

A set of attributes that bears on the ability of software to be transferred from one environment to another, without minimum changes.
Required functions are:



Portability

Characteristics of "Software" in Software Engineering

1. **Software is developed or engineered; it is not manufactured in the classical sense:**
    - Although some similarities exist between software development and hardware manufacturing, few activities are fundamentally different.
    - In both activities, high quality is achieved through good design, but the manufacturing phase for hardware can introduce quality problems than software.
2. **The software doesn't "wear out.":**
    - Hardware components suffer from the growing effects of many other environmental factors. Stated simply, the hardware begins to wear out.
    - Software is not susceptible to the environmental maladies that cause hardware to wear out.
    - When a hardware component wears out, it is replaced by a spare part.
    - There are no software spare parts.
    - Every software failure indicates an error in design or in the process through which the design was translated into machine-executable code. Therefore, the software maintenance tasks that accommodate requests for change involve considerably more complexity than hardware maintenance. However, the implication is clear—the software doesn't wear out. But it does deteriorate.
3. **The software continues to be custom-built:**
    - A software part should be planned and carried out with the goal that it tends to be reused in various projects.
    - Current reusable segments encapsulate the two pieces of information and the preparation that is applied to the information, empowering the programmer to make new applications from reusable parts.

- In the hardware world, component reuse is a natural part of the engineering process.

Characteristics of the Software

- It is intangible, meaning it cannot be seen or touched.
- It is non-perishable, meaning it does not degrade over time.
- It is easy to replicate, meaning it can be copied and distributed easily.
- It can be complex, meaning it can have many interrelated parts and features.
- It can be difficult to understand and modify, especially for large and complex systems.
- It can be affected by changing requirements, meaning it may need to be updated or modified as the needs of users change.
- It can be impacted by bugs and other issues, meaning it may need to be tested and debugged to ensure it works as intended.

Software Crisis – Software Engineering

The term "software crisis" refers to the numerous challenges and difficulties faced by the software industry during the 1960s and 1970s. It became clear that old methods of developing software couldn't keep up with the growing complexity and demands of new projects. This led to high costs, delays, and poor-quality software. New methodologies and tools were needed to address these issues.

What is Software Crisis?

Software Crisis is a term used in computer science for the difficulty of writing useful and efficient computer programs in the required time. The software crisis was due to using the same workforce, same methods, and same tools even though rapidly increasing software demand, the complexity of software, and software challenges. With the increase in software complexity, many software problems arose because existing methods were insufficient.

Suppose we use the same workforce, same methods, and same tools after the fast increase in software demand, software complexity, and software challenges. In that case, there arise some issues like software budget problems, software efficiency problems, software quality problems, software management, and delivery problems, etc. This condition is called a **Software Crisis.**

Causes of Software Crisis

Following are the causes of Software Crisis:

- The cost of owning and maintaining software was as expensive as developing the software.
- At that time Projects were running overtime.
- At that time Software was very inefficient.
- The quality of the software was low quality.
- Software often did not meet user requirements.
- The average software project overshoots its schedule by half.
- At that time Software was never delivered.
- Non-optimal resource utilization.
- Challenging to alter, debug, and enhance

Factors Contributing to Software Crisis
Factor Contributing to Software Crisis are:

- Poor project management.
- Lack of adequate training in software engineering.
- Less skilled project members.
- Low productivity improvements.

Solution of Software Crisis:

There is no single solution to the crisis. One possible solution to a software crisis is Software Engineering because software engineering is a systematic, disciplined, and quantifiable approach. For preventing software crises, there are some guidelines:

- Reduction in software over budget.
- The quality of the software must be high.
- Less time is needed for a software project.
- Experienced and skilled people working on the software project.
- Software must be delivered.
- Software must meet user requirements.

Software Process Model in Software Engineering

The term **software** specifies to the set of computer programs, procedures and associated documents (Flowcharts, manuals, etc.) that describe the program and how they are to be used. A software process is the set of activities and associated outcome that produce a software product. Software engineers mostly carry out these activities. These are four key process activities, which are common to all software processes. These activities are:

1. **Software specifications:** The functionality of the software and constraints on its operation must be defined.
2. **Software development:** The software to meet the requirement must be produced.
3. **Software validation:** The software must be validated to ensure that it does what the customer wants.
4. **Software evolution:** The software must evolve to meet changing client needs.

The Software Process Model

A software process model is a specified definition of a software process, which is presented from a particular perspective. Models, by their nature, are a simplification, so a software process model is an abstraction of the actual process, which is being described. Process models may contain activities, which are part of the software process, software product, and the roles of people involved in software engineering. Some examples of the types of software process models that may be produced are:

1. **A workflow model:** This shows the series of activities in the process along with their inputs, outputs and dependencies. The activities in this model perform human actions.
2. **2. A dataflow or activity model:** This represents the process as a set of activities, each of which carries out some data transformations. It shows how the input to the process, such as a specification is converted to an output such as a design. The activities here

may be at a lower level than activities in a workflow model. They may perform transformations carried out by people or by computers.

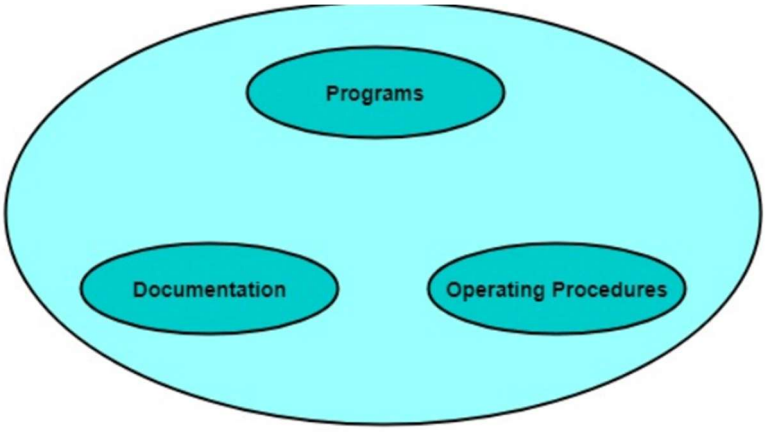
3. **3. A role/action model:** This means the roles of the people involved in the software process and the activities for which they are responsible.

There are several various general models or paradigms of software development:

1. **The waterfall approach:** This takes the above activities and produces them as separate process phases such as requirements specification, software design, implementation, testing, and so on. After each stage is defined, it is "signed off" and development goes onto the following stage.
2. **Evolutionary development:** This method interleaves the activities of specification, development, and validation. An initial system is rapidly developed from a very abstract specification.
3. **Formal transformation:** This method is based on producing a formal mathematical system specification and transforming this specification, using mathematical methods to a program. These transformations are 'correctness preserving.' This means that you can be sure that the developed programs meet its specification.
4. **System assembly from reusable components:** This method assumes the parts of the system already exist. The system development process target on integrating these parts rather than developing them from scratch.

Program vs. Software

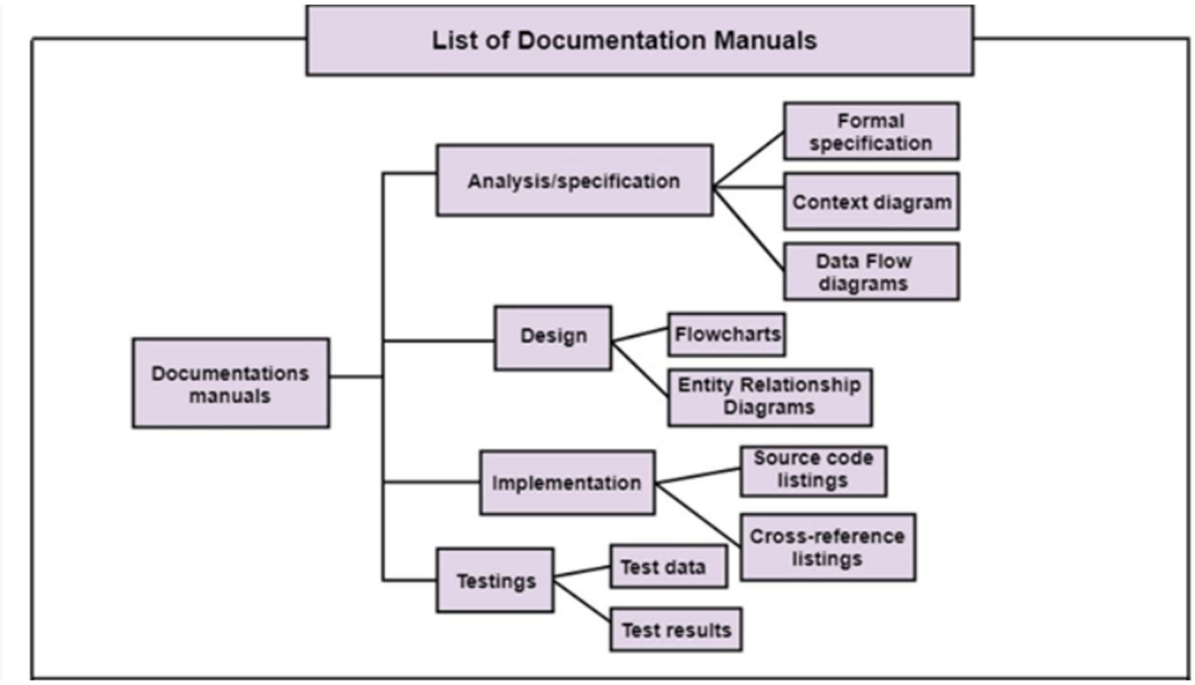
Software is more than programs. Any program is a subset of software, and it becomes software only if documentation & operating procedures manuals are prepared.
There are three components of the software as shown in fig:



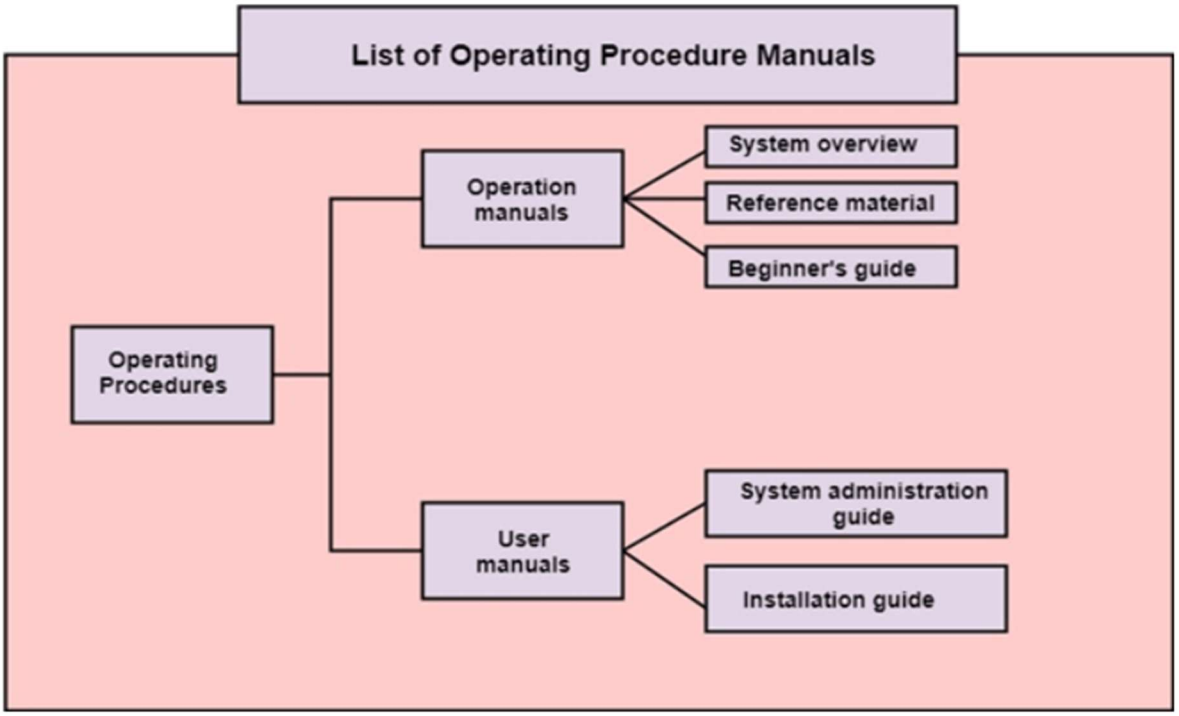
Software= Program + Documentation + Operating Procedures

Fig:Components of Software

1. Program: Program is a combination of source code & object code.
2. Documentation: Documentation consists of different types of manuals. Examples of documentation manuals are: Data Flow Diagram, Flow Charts, ER diagrams, etc.



3. Operating Procedures: Operating Procedures consist of instructions to set up and use the software system and instructions on how react to the system failure. Example of operating system procedures manuals is: installation guide, Beginner's guide, reference guide, system administration guide, etc.



Difference Between Software Engineering Process and Conventional Engineering Process

Aspect	Software Engineering Process	Conventional Engineering Process
Process	Software Engineering Process is a process that majorly involves computer science, information technology, and discrete mathematics.	The conventional Engineering Process is a process that majorly involves science, mathematics, and empirical knowledge.
Focus Area	It is mainly related to computers, programming, and writing codes for building applications.	It is about building cars, machines, hardware, buildings, etc.
Cost	In Software Engineering Process construction and development costs are low.	In Conventional Engineering Process construction and development cost is high.
Application	It can involve the application of new and untested elements in software projects.	It usually applies only known and tested principles to meet product requirements.
Development Effort	In the Software Engineering Process, most development effort goes into building new designs and features.	In Conventional Engineering Process, most development efforts are required to change old designs.
Emphasis	It majorly emphasizes quality.	It majorly emphasizes mass production.
Product Nature	Product development develops intangible products (software).	Product development develops tangible products (e.g. bridges, buildings).
Design Requirement	Design requirements may change throughout the development process.	Design Requirements are typically well-defined upfront.
Testing	Testing is an integral part of the development process.	Testing occurs mainly after product completion.
Prototyping	Prototyping is common and helps to refine requirements.	Prototyping is less common due to cost and time.

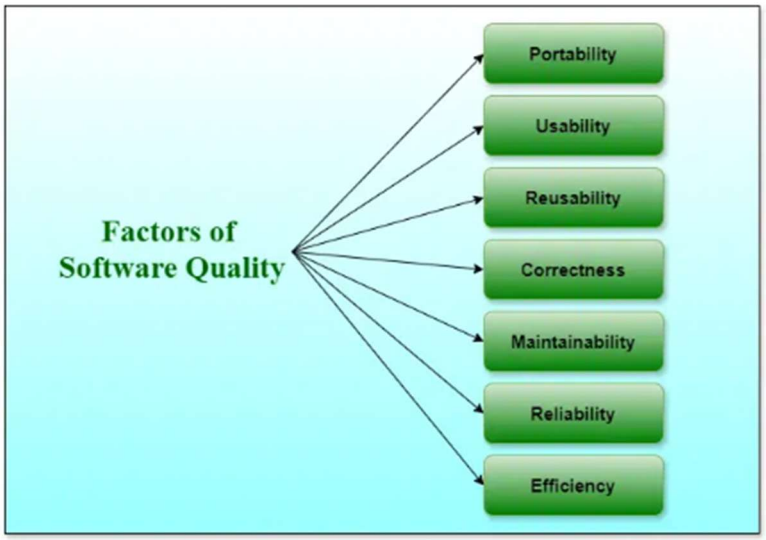
Aspect	Software Engineering Process	Conventional Engineering Process
Maintenance	Maintenance and updates are necessary to keep software relevant.	Maintenance is typically scheduled or reactive.
Complexity	Software development often involves complex logic and algorithms.	Conventional engineering may have more complex physical properties to deal with.
Framework	Software development often follows established standards and frameworks.	Conventional engineering may have well-established regulations and standards.
Cost Dynamics	Software development is typically less expensive to start, but costs may increase with maintenance and updates.	Conventional engineering may be more expensive to start due to materials and construction but may have lower maintenance costs.
Methodologies	Agile methodologies are commonly used in software development.	Conventional engineering may use more traditional project management approaches.

Software Quality – Software Engineering

Traditionally, a high-quality product is outlined in terms of its fitness of purpose. That is, a high-quality product will specifically be what the users need to try. For code products, the fitness of purpose is typically taken in terms of satisfaction of the wants arranged down within the SRS document.

What is Software Quality?
Software Quality shows how good and reliable a product is. To convey an associate degree example, think about functionally correct software. It performs all functions as laid out in the SRS document. But it has an associate degree virtually unusable program. even though it should be functionally correct, we tend not to think about it to be a high-quality product.
Another example is also that of a product that will have everything that the users need but has an associate degree virtually incomprehensible and not maintainable code. Therefore, the normal construct of quality as “fitness of purpose” for code products isn’t satisfactory.

Factors of Software Quality
The modern read of high-quality associates with software many quality factors like the following:



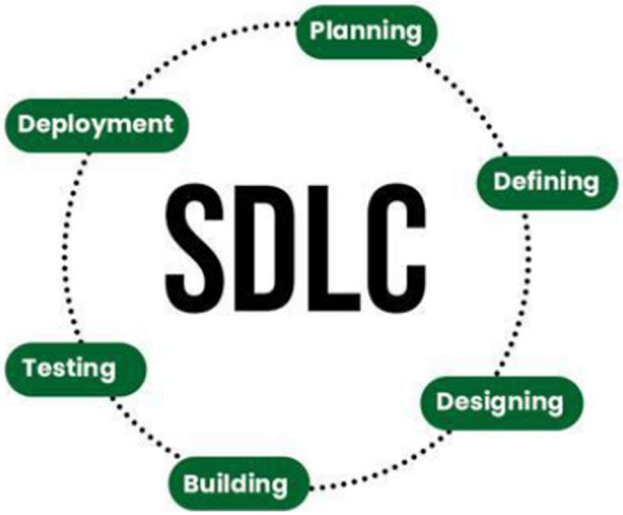
1. **Portability:** A software is claimed to be transportable, if it may be simply created to figure in several package environments, in several machines, with alternative code products, etc.
2. **Usability:** A software has smart usability if completely different classes of users (i.e. knowledgeable and novice users) will simply invoke the functions of the products.
3. **Reusability:** A software has smart reusability if completely different modules of the products will simply be reused to develop new products.
4. **Correctness:** Software is correct if completely different needs as laid out in the SRS document are properly enforced.
5. **Maintainability:** A software is reparable, if errors may be simply corrected as and once they show up, new functions may be simply added to the products, and therefore the functionalities of the products may be simply changed, etc
6. **Reliability:** Software is more reliable if it has fewer failures. Since software engineers do not deliberately plan for their software to fail, reliability depends on the number and type of mistakes they make. Designers can improve reliability by ensuring the software is easy to implement and change, by testing it thoroughly, and also by ensuring that if failures occur, the system can handle them or can recover easily.
7. **Efficiency.** The more efficient software is, the less it uses of CPU-time, memory, disk space, network bandwidth, and other resources. This is important to customers in order to reduce their costs of running the software, although with today's powerful computers, CPU time, memory and disk usage are less of a concern than in years gone by.

Software Development Life Cycle (SDLC)

oftware development life cycle (SDLC) is a structured process that is used to design, develop, and test good-quality software. SDLC, or software development life cycle, is a methodology that defines the entire procedure of software development step-by-step. The **goal of the SDLC life cycle model** is to deliver high-quality, maintainable software that meets the user's requirements. SDLC in software engineering models outlines the plan for each stage so that each stage of the software development model can perform its task efficiently to deliver the software at a low cost within a given time frame that meets users' requirements. In this article we will see Software Development Life Cycle (SDLC) in detail.

What is the Software Development Life Cycle (SDLC)?

SDLC is a process followed for software building within a software organization. SDLC consists of a precise plan that describes how to develop, maintain, replace, and enhance specific software. The life cycle defines a method for improving the quality of software and the all-around development process.



Stages of the Software Development Life Cycle
SDLC specifies the task(s) to be performed at various stages by a software engineer or developer. It ensures that the end product is able to meet the customer's expectations and fits within the overall budget. Hence, it's vital for a software developer to have prior knowledge of this software development process. SDLC is a collection of these six stages, and the stages of SDLC are as follows:

Stage-1: Planning and Requirement Analysis
Planning is a crucial step in everything, just as in software development. In this same stage, requirement analysis is also performed by the developers of the organization. This is attained from customer inputs, and sales department/market surveys.
The information from this analysis forms the building blocks of a basic project. The quality of the project is a result of planning. Thus, in this stage, the basic project is designed with all the available information.

Stage-2: Defining Requirements
In this stage, all the requirements for the target software are specified. These requirements get approval from customers, market analysts, and stakeholders.

This is fulfilled by utilizing SRS (Software Requirement Specification). This is a sort of document that specifies all those things that need to be defined and created during the entire project cycle.

Stage-3: Designing Architecture
SRS is a reference for software designers to come up with the best architecture for the software. Hence, with the requirements defined in SRS, multiple designs for the product architecture are

present in the Design Document Specification (DDS).
This DDS is assessed by market analysts and stakeholders. After evaluating all the possible factors, the most practical and logical design is chosen for development.

Stage-4: Developing Product

At this stage, the fundamental development of the product starts. For this, developers use a specific programming code as per the design in the DDS. Hence, it is important for the coders to follow the protocols set by the association. Conventional programming tools like compilers, interpreters, debuggers, etc. are also put into use at this stage. Some popular languages like C/C++, Python, Java, etc. are put into use as per the software regulations.

Stage-5: Product Testing and Integration

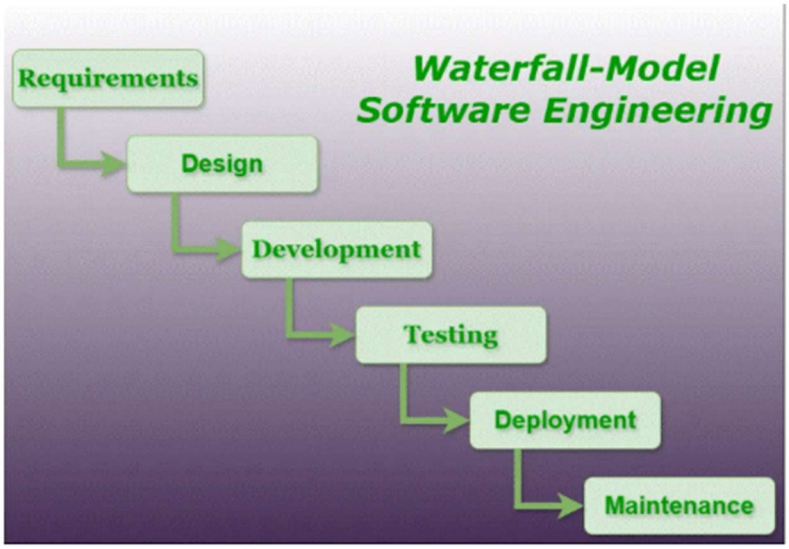
After the development of the product, testing of the software is necessary to ensure its smooth execution. Although, minimal testing is conducted at every stage of SDLC. Therefore, at this stage, all the probable flaws are tracked, fixed, and retested. This ensures that the product confronts the quality requirements of SRS.
Documentation, Training, and Support: Software documentation is an essential part of the software development life cycle. A well-written document acts as a tool and means to information repository necessary to know about software processes, functions, and maintenance. Documentation also provides information about how to use the product. Training in an attempt to improve the current or future employee performance by increasing an employee's ability to work through learning, usually by changing his attitude and developing his skills and understanding.

Stage-6: Deployment and Maintenance of Products

After detailed testing, the conclusive product is released in phases as per the organization's strategy. Then it is tested in a real industrial environment. It is important to ensure its smooth performance. If it performs well, the organization sends out the product as a whole. After retrieving beneficial feedback, the company releases it as it is or with auxiliary improvements to make it further helpful for the customers. However, this alone is not enough. Therefore, along with the deployment, the product's supervision.

Phases of Waterfall Model

Classical Waterfall Model divides the life cycle into a set of phases. The development process can be considered as a sequential flow in the waterfall. The different sequential phases of the classical waterfall model are follow:



Waterfall Model-Software Engineering

1. Requirements Analysis and Specification

Requirement Analysis and specification phase aims to understand the exact requirements of the customer and document them properly. This phase consists of two different activities.

1. Requirement Gathering and Analysis: Firstly, all the requirements regarding the software are gathered from the customer and then the gathered requirements are analysed.
The goal of the analysis part is to remove incompleteness (an incomplete requirement is one in which some parts of the actual requirements have been omitted) and inconsistencies (an inconsistent requirement is one in which some part of the requirement contradicts some other part).

2. Requirement Specification: These analysed requirements are documented in a software requirement specification (SRS) document. SRS document serves as a contract between the development team and customers. Any future dispute between the customers and the developers can be settled by examining the SRS document.

2. Design

The goal of this **Software Design Phase** is to convert the requirements acquired in the SRS into a format that can be coded in a programming language. It includes high-level and detailed design as well as the overall software architecture. A **Software Design Document** is used to document all of this effort (SDD).

- **High-Level Design (HLD)**: This phase focuses on outlining the broad structure of the system. It highlights the key components and how they interact with each other, giving a clear overview of the system's architecture.
- **Low-Level Design (LLD)**: Once the high-level design is in place, this phase zooms into the details. It breaks down each component into smaller parts and provides specifics about how each part will function, guiding the actual coding process.

4. Development

In the **Development Phase** software design is translated into source code using any suitable programming language. Thus each designed module is coded. The unit testing phase aims to check whether each module is working properly or not.

- In this phase, developers begin writing the actual source code based on the designs created earlier.
- The goal is to transform the design into working code using the most suitable programming languages.
- Unit tests are often performed during this phase to make sure that each component functions correctly on its own.

5. Testing and Deployment

Testing: Integration of different modules is undertaken soon after they have been coded and unit tested. Integration of various modules is carried out incrementally over several steps. During each integration step, previously planned modules are added to the partially integrated system and the resultant system is tested. Finally, after all the modules have been successfully integrated and tested, the full working system is obtained and system testing is carried out on this. System testing consists of three different kinds of testing activities as described below.

- **Alpha testing:** Alpha testing is the system testing performed by the development team.
- **Beta testing:** Beta testing is the system testing performed by a friendly set of customers.
- **Acceptance testing:** After the software has been delivered, the customer performs acceptance testing to determine whether to accept the delivered software or reject it.

6. Maintenance

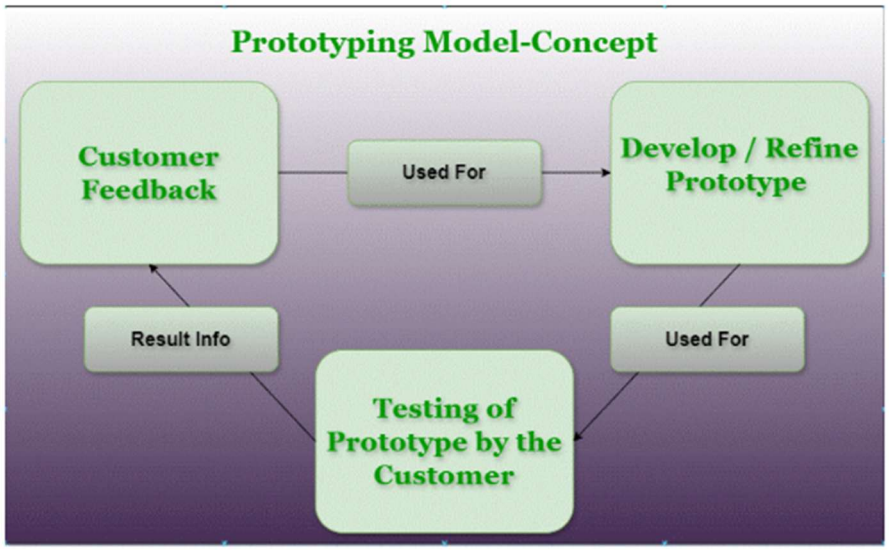
In **Maintenance Phase** is the most important phase of a software life cycle. The effort spent on maintenance is 60% of the total effort spent to develop a full software. There are three types of maintenance.

- **Corrective Maintenance:** This type of maintenance is carried out to correct errors that were not discovered during the product development phase.
- **Perfective Maintenance:** This type of maintenance is carried out to enhance the functionalities of the system based on the customer's request.
- **Adaptive Maintenance:** Adaptive maintenance is usually required for porting the software to work in a new environment such as working on a new computer platform or with a new operating system.

What is Prototyping Model?

The Prototyping Model is one of the most popularly used **Software Development Life Cycle Models (SDLC models)**. This model is used when the customers do not know the exact project requirements beforehand.

In this model, a prototype of the end product is first developed, tested, and refined as per customer feedback repeatedly till a final acceptable prototype is achieved which forms the basis for developing the final product.



Prototyping Model-Concept

In this process model, the system is partially implemented before or during the analysis phase thereby allowing the customers to see the product early in the life cycle. The process starts by interviewing the customers and developing the incomplete high-level paper model.
This document is used to build the initial prototype supporting only the basic functionality as desired by the customer. Once the customer figures out the problems, the prototype is further refined to eliminate them. The process continues until the user approves the prototype and finds the working model to be satisfactory.

Advantages of Prototyping Model

- The customers get to see the partial product early in the life cycle. This ensures a greater level of customer satisfaction and comfort.
- New requirements can be easily accommodated as there is scope for refinement.
- Missing functionalities can be easily figured out.
- Errors can be detected much earlier thereby saving a lot of effort and cost, besides enhancing the quality of the software.
- The developed prototype can be reused by the developer for more complicated projects in the future.
- Flexibility in design.
- Early feedback from customers and stakeholders can help guide the development process and ensure that the final product meets their needs and expectations.
- Prototyping can be used to test and validate design decisions, allowing for adjustments to be made before significant resources are invested in development.
- Prototyping can help reduce the risk of project failure by identifying potential issues and addressing them early in the process.
- Prototyping can facilitate communication and collaboration among team members and stakeholders, improving overall project efficiency and effectiveness.

- Prototyping can help bridge the gap between technical and non-technical stakeholders by providing a tangible representation of the product.

Disadvantages of the Prototyping Model

- Costly concerning time as well as money.
- There may be too much variation in requirements each time the prototype is evaluated by the customer.
- Poor Documentation due to continuously changing customer requirements.
- It is very difficult for developers to accommodate all the changes demanded by the customer.
- There is uncertainty in determining the number of iterations that would be required before the prototype is finally accepted by the customer.
- After seeing an early prototype, the customers sometimes demand the actual product to be delivered soon.
- Developers in a hurry to build prototypes may end up with sub-optimal solutions.
- The customer might lose interest in the product if he/she is not satisfied with the initial prototype.
- The prototype may not be scalable to meet the future needs of the customer.
- The prototype may not accurately represent the final product due to limited functionality or incomplete features.
- The focus on prototype development may shift away from the final product, leading to delays in the development process.
- The prototype may give a false sense of completion, leading to the premature release of the product.
- The prototype may not consider technical feasibility and scalability issues that can arise during the final product development.
- The prototype may be developed using different tools and technologies, leading to additional training and maintenance costs.
- The prototype may not reflect the actual business requirements of the customer, leading to dissatisfaction with the final product.

Applications of Prototyping Model

- The Prototyping Model should be used when the requirements of the product are not clearly understood or are unstable.
- The prototyping model can also be used if requirements are changing quickly.
- This model can be successfully used for developing user interfaces, high-technology software-intensive systems, and systems with complex algorithms and interfaces.
- The prototyping Model is also a very good choice to demonstrate the technical feasibility of the product.

What is Spiral Model in Software Engineering?

The Spiral Model is one of the most important SDLC model. The Spiral Model is a combination of the waterfall model and the iterative model. It provides support for **Risk Handling**.
The Spiral Model was first proposed by **Barry Boehm**. This article focuses on discussing the Spiral Model in detail.

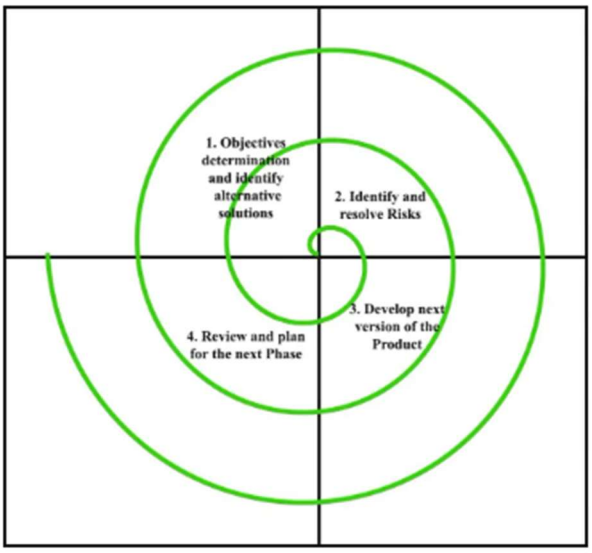
The Spiral Model is a **Software Development Life Cycle (SDLC)** model that provides a systematic and iterative approach to software development. In its diagrammatic representation, looks like a spiral with many loops. The exact number of loops of the spiral is unknown and can vary from project to project. Each loop of the spiral is called a **phase** of the software development process.

Some **Key Points** regarding the **Stages of a Spiral Model**:

1. The exact number of phases needed to develop the product can be varied by the project manager depending upon the project risks.
2. As the project manager dynamically determines the number of phases, the project manager has an important role in developing a product using the spiral model.
3. It is based on the idea of a spiral, with each iteration of the spiral representing a complete software development cycle, from requirements gathering and analysis to design, implementation, testing, and maintenance.

Phases of the Spiral Model

The Spiral Model is a risk-driven model, meaning that the focus is on managing risk through multiple iterations of the software development process. Each phase of the Spiral Model is divided into four Quadrants:



Spiral Model – SDLC

1. Objectives Defined

In first phase of the spiral model, we clarify what the project aims to achieve, including functional and non-functional requirements.
Requirements are gathered from the customers and the objectives are identified, elaborated, and analyzed at the start of every phase. Then alternative solutions possible for the phase are proposed in this quadrant.

2. Risk Analysis and Resolving

In the risk analysis phase, the risks associated with the project are identified and evaluated.
During the second quadrant, all the possible solutions are evaluated to select the best possible solution. Then the risks associated with that solution are identified and the risks are resolved using the best possible strategy. At the end of this quadrant, the Prototype is built for the best possible solution.

3. Develop the next version of the Product
During the third quadrant, the identified features are developed and verified through testing. At the end of the third quadrant, the next version of the software is available.
In the evaluation phase, the software is evaluated to determine if it meets the customer's requirements and if it is of high quality.

4. Review and plan for the next Phase

In the fourth quadrant, the Customers evaluate the so-far developed version of the software. In the end, planning for the next phase is started.
The next iteration of the spiral begins with a new planning phase, based on the results of the evaluation.
The Spiral Model is often used for complex and large software development projects, as it allows for a more flexible and adaptable approach to Software development. It is also well-suited to projects with significant uncertainty or high levels of risk.

Advantages of the Spiral Model

Below are some advantages of the Spiral Model.

1. **Risk Handling:** The projects with many unknown risks that occur as the development proceeds, in that case, Spiral Model is the best development model to follow due to the risk analysis and risk handling at every phase.
2. **Good for large projects:** It is recommended to use the Spiral Model in large and complex projects.
3. **Flexibility in Requirements:** Change requests in the Requirements at a later phase can be incorporated accurately by using this model.
4. **Customer Satisfaction:** Customers can see the development of the product at the early phase of the software development and thus, they habituated with the system by using it before completion of the total product.
5. **Iterative and Incremental Approach:** The Spiral Model provides an iterative and incremental approach to software development, allowing for flexibility and adaptability in response to changing requirements or unexpected events.
6. **Emphasis on Risk Management:** The Spiral Model places a strong emphasis on risk

management, which helps to minimize the impact of uncertainty and risk on the software development process.

7. **Improved Communication:** The Spiral Model provides for regular evaluations and reviews, which can improve communication between the customer and the development team.
8. **Improved Quality:** The Spiral Model allows for multiple iterations of the software development process, which can result in improved software quality and reliability.

Disadvantages of the Spiral Model

Below are some main disadvantages of the spiral model.

1. Complex: The Spiral Model is much more complex than other SDLC models.
2. Expensive: Spiral Model is not suitable for small projects as it is expensive.
3. Too much dependability on Risk Analysis: The successful completion of the project is very much dependent on Risk Analysis. Without very highly experienced experts, it is going to be a failure to develop a project using this model.
4. Difficulty in time management: As the number of phases is unknown at the start of the project, time estimation is very difficult.
5. Complexity: The Spiral Model can be complex, as it involves multiple iterations of the software development process.
6. Time-Consuming: The Spiral Model can be time-consuming, as it requires multiple evaluations and reviews.
7. Resource Intensive: The Spiral Model can be resource-intensive, as it requires a significant investment in planning, risk analysis, and evaluations.

Evolutionary Model – Software Engineering

The evolutionary model is a combination of the Iterative and Incremental models of the software development life cycle. Delivering your system in a big bang release, delivering it in incremental process over time is the action done in this model. Some initial requirements and architecture envisioning need to be done. It is better for software products that have their feature sets redefined during development because of user feedback and other factors. This article focuses on discussing the Evolutionary Model in detail.

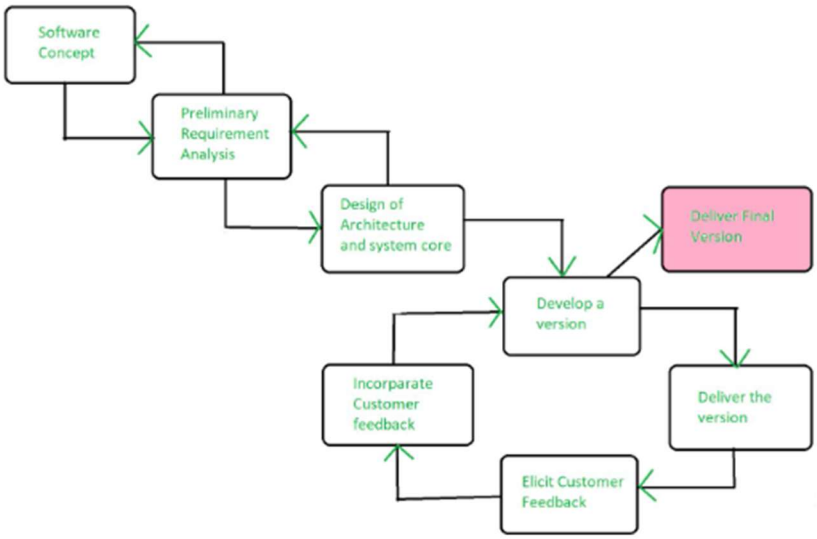
What is the Evolutionary Model?

The Evolutionary development model divides the development cycle into smaller, incremental waterfall models in which users can get access to the product at the end of each cycle.

1. Feedback is provided by the users on the product for the planning stage of the next cycle and the development team responds, often by changing the product, plan, or process.
2. Therefore, the software product evolves with time.
3. All the models have the disadvantage that the duration of time from the start of the project to the delivery time of a solution is very high.
4. The evolutionary model solves this problem with a different approach.
5. The evolutionary model suggests breaking down work into smaller chunks, prioritizing them, and then delivering those chunks to the customer one by one.
6. The number of chunks is huge and is the number of deliveries made to the customer.
7. The main advantage is that the customer's confidence increases as he constantly gets quantifiable goods or services from the beginning of the project to verify and validate

his requirements.

8. The model allows for changing requirements as well as all work is broken down into maintainable work chunks.



Evolutionary Model

Application of Evolutionary Model

1. It is used in large projects where you can easily find modules for incremental implementation. Evolutionary model is commonly used when the customer wants to start using the core features instead of waiting for the full software.
2. Evolutionary model is also used in object-oriented software development because the system can be easily portioned into units in terms of objects.

Necessary Conditions for Implementing this Model

1. Customer needs are clear and been explained in deep to the developer team.
2. There might be small changes required in separate parts but not a major change.
3. As it requires time, so there must be some time left for the market constraints.
4. Risk is high and continuous targets to achieve and report to customer repeatedly.
5. It is used when working on a technology is new and requires time to learn.

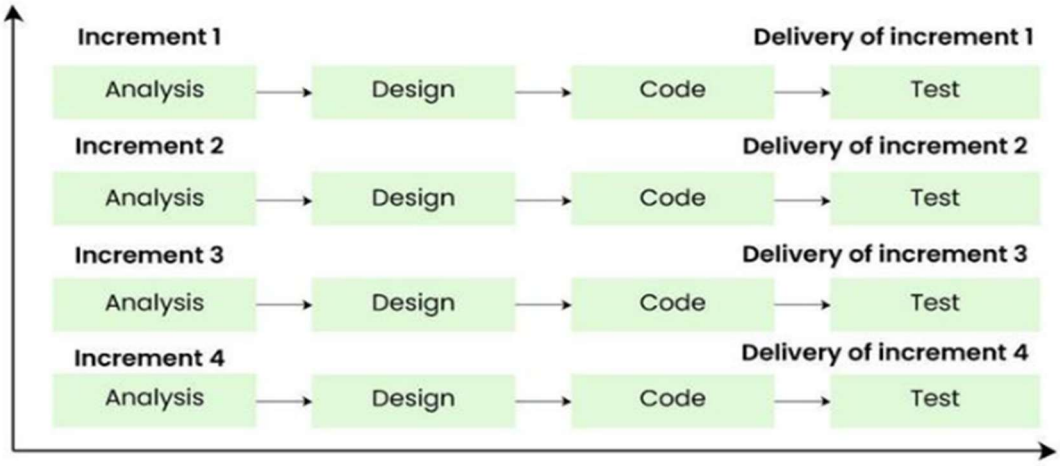
Advantages Evolutionary Model

1. **Adaptability to Changing Requirements:** Evolutionary models work effectively in projects when the requirements are ambiguous or change often. They support adjustments and flexibility along the course of development.
2. **Early and Gradual Distribution:** Functional components or prototypes can be delivered early thanks to incremental development. Faster user satisfaction and feedback may result from this.
3. **User Commentary and Involvement:** Evolutionary models place a strong emphasis on ongoing user input and participation. This guarantees that the software offered closely matches the needs and expectations of the user.
4. **Improved Handling of Difficult Projects:** Big, complex tasks can be effectively

managed with the help of evolutionary models. The development process is made simpler by segmenting the project into smaller, easier-to-manage portions.

Iterative Enhancement Model in Software Development

In software development, the Iterative Enhancement Model stands out due to its incremental and iterative nature, it is also known as an **incremental model**. This approach focuses on incremental development and improvement rather than trying to complete a software product in a single phase. This model is based on segmenting the project into smaller units, or iterations, with a set of tasks completed.



Advantages of Iterative Enhancement Model

- Adaptation to changing requirements is made possible by its flexibility in accommodating modifications and improvement during each iteration.
- Early software iterations provide clients with functional portions of the product, facilitating prompt feedback and validation.
- Problems and risks can be identified and addressed early in the development process, reduces chances of issue for future stages.
- Feedback and constant client involvement are encouraged to make sure the finished product lives up to user expectations.
- Every iteration is put through testing and improvement, leading to higher quality product.

Disadvantages of Iterative Enhancement Model

- Especially in larger projects, managing several iterations at once can add complexity.
- Higher cost
- Due to constant changes, there may be delays in documentation, making it more difficult to maintain comprehensive documentation.
- Continuous customer engagement may not be possible in all scenarios, which impacts the effectiveness of the model.